

```
class Banner
```

```
attr_accessible :horiz, :link, :visible, :image, :position  
has_attached_file :image, styles: { vert: '220'
```

```
before_create :assign_position
```

```
class Banner < ActiveRecord::Base
```

```
attr_accessible :horiz, :link, :visible, :image  
has_attached_file :image, styles: { vert: '220'
```

```
before_create :assign_position
```

```
protected
```

```
def assign_position
```

```
max = Banner.maximum(:position)
```

```
self.position = max ? max + 1 : 0
```

FULL STACK DEVELOPMENT

NEXT.JS

AULA 05

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	15
O QUE VOCÊ VIU NESTA AULA?	24
REFERÊNCIAS.....	25

EMSE

O QUE VEM POR AÍ?

Nesta aula, você aprenderá como funciona e como implementar BFF em nossa aplicação. Vamos recapitular nossos componentes para analisarmos como efetuamos o uso de BFF em vários pontos, nos aprofundaremos nas possibilidades e aprenderemos como aplicar no nosso cenário!

EMSE

HANDS ON

Nesta aula, abordaremos o uso de BFF na nossa aplicação. Veremos como a implementação de BFF é simples, os recursos diferentes que são disponíveis com o uso e as vantagens que obtemos ao utilizá-lo.

Também aprenderemos como utilizar o socket-io em nossa aplicação e criaremos um hook para nos auxiliar no desenvolvimento da nossa aplicação.

Abaixo temos os comandos usados na aula:

```
npm i socket.io-client
```

A seguir temos o código da aula:

UseCard.tsx

```
import { MessageSquareMore } from 'lucide-react';
import { useRouter } from 'next/router';
import { useEffect, useState } from 'react';

export const UserCard = ({ data }: { data: any }) => {
  const [userName, setUserName] = useState("");
  const [id, setId] = useState(0);
  const router = useRouter()
  useEffect(() => {
    const getCookieValue = (name:any) => (
      document.cookie.split('; ').find(row => row.startsWith(`${name}=`)).split('=')[1]
    );
    setUserName(getCookieValue('userName') ?? "");
    setId(parseInt(getCookieValue('id') ?? ""));
  }, []);
```

```

const handleClick = () => {
  router.push({
    pathname: `/chat/${data.name}`,
    query: { recipientId: data.id, senderId: id, senderName: userName },
  });
};

return(
  <div className='flex flex-1 flex-row font-sans justify-around items-center bg-background-800 w-3/4 max-h-10 mt-2 mb-2 rounded-xl hover:opacity-80 cursor-pointer'
    onClick={handleClick}>
    <span className='flex flex-1 ml-10 w-2/4'>{data.name}</span>
    <div className="flex flex-1 justify-center w-2/4">
      <MessageSquareMore className='h-8 w-8 text-fontColor-900'/>
    </div>
  </div>
)
}

```

[destinatário].tsx

```

import { FormEvent, useEffect, useRef, useState } from 'react';
import { useRouter } from 'next/router';
import { Send } from 'lucide-react';
import useSocket from '@/hooks/useSocket';
import { Message } from '@app/components/Message';

```

```
interface IMessage {  
  name: string;  
  content: string;  
  idRemetente: number;  
  idDestinatario: number;  
}  
  
export async function getServerSideProps(context) {  
  const { params, query } = context ;  
  const recipientId = query.recipientId;  
  const senderId = query.senderId;  
  const senderName = query.senderName;  
  try {  
    const response = await  
fetch(`http://localhost:3333/api/users/id?id=${recipientId}`, {  
  method: 'GET',  
  headers: {  
    'Content-Type': 'application/json',  
  },  
});  
  if (response.status === 200) {  
    const userSender = await response.json();  
    return { props: { userSender, senderId, senderName } };  
  } else {  
    return { props: { userSender:"" } };  
  }  
}
```

```

    }
  } catch (error) {
    console.error('Erro ao buscar dados:', error);
    return { props: { userSender: null } };
  }
}

export default function ChatPage({userSender, senderId, senderName}) {
  const router = useRouter();
  const { recipientId } = router.query;
  const [id, setId] = useState(senderId);
  const [input, setInput] = useState("");
  const [messages, setMessages] = useState<IMessage[]>([]);
  const { socketInstance } = useSocket(senderId);
  const messageContainer = useRef<HTMLDivElement>(null);

  useEffect(() => {
    messageContainer.current?.scrollTo(
      0,
      messageContainer.current.scrollHeight
    );
  }, [messages]);

  const handleSubmit = (event) => {
    event.preventDefault();
  }
}

```

```

const newMessage: IMessage = {
  content: input,
  name: senderName,
  idRemetente: parseInt(id) as number,
  idDestinatario: userSender?.id as number,
};

console.log('Enviando mensagem:', newMessage);

setInput("");
socketInstance.emit('message', newMessage);
setMessages((previous) => [...previous, newMessage]);
};

useEffect(() => {
  socketInstance?.on('message', (message: IMessage) => {
    console.log('Mensagem recebida:', message);
    if (message.idDestinatario === userSender?.id) {
      setMessages((previous) => [...previous, message]);
    }
  });
});

}, [socketInstance, userSender]);

useEffect(() => {
  function onMessageEvent(value: IMessage) {
    setMessages((previous) => [...previous, value]);
  }
}

```



```

function onNotificationEvent(value: IMessage) {
  console.log("hello");
}

socketInstance.on("notification", onNotificationEvent);
socketInstance.on("message", onMessageEvent);
return () => {
  socketInstance.off("message", onMessageEvent);
  socketInstance.off("notification", onNotificationEvent);
};
}, [socketInstance]);

useEffect(() => {
  if (id !== null) {
    socketInstance.emit('historicoMensagens', {
      idRemetente: parseInt(id) as number,
      idDestinatario: userSender?.id as number,
    });

    socketInstance.on('historicoMensagens', (historyMessages) => {
      setMessages(historyMessages);
    });
  }
}, []);

```

```

return (

  <main className="content-area w-[100vw] h-[100vh] flex flex-col items-center
  justify-center bg-background-900">

    <h6 className='font-sans text-2xl text-fontColor-900'>Chat com:
    {userSender?.name}</h6>

    <div ref={messageContainer} className='flex h-[50rem] w-[60rem] flex-col
    overflow-auto px-5 pt-14 bg-background-800'>

      {messages.map((message,index) => (

        <Message

          key={index}

          message={message.content}

          isOwner={(message.idRemetente === parseInt(id)) ? true : false}

          data={message}

        />

      ))}

    </div>

    <form

      onSubmit={handleSubmit}

      className="flex justify-between border border-fontColor-900 rounded-xl w-
      [60rem] bg-background-800"

      >

      <input

        type="text"

        placeholder='Digite sua mensagem'

        onChange={(event) => setInput(event.target.value)}

        value={input}

```

```

        className="h-10 w-full rounded-bl-[9px] rounded-tl-[10px] bg-background-800
px-4 py-1 text-3xl text-fontColor-900 focus:outline-none"

    />

    <button

        type="submit"

        className="rounded-br-[9px] rounded-tr-[10px] bg-background-800 px-4 py-
1"

    >

        <Send className="h-8 w-8 text-fontColor-900 hover:text-white-1" size={34}
/>

    </button>

</form>

</main>

);
}

```

Message.tsx

```

interface IMessageProps {

    message: any;

    data: any

    isOwner?: boolean;

}

export const Message = ({ message, isOwner = false, data }: IMessageProps) => {

    const positionTriangle = isOwner ? "after:right-[-6px] rounded-br-[0px]" : "after:left-[-
6px] rounded-bl-[0px]";

    const MessageParagraph = (

        <p

```

```

      className={`triangle relative h-fit rounded-xl bg-fontColor-900 px-6 py-2
after:bottom-0 ${positionTriangle}`}
    >
      {message}
    </p>
  );

  return (
    <div className={`m-2 flex ${isOwner && "self-end"}`} >
      {isOwner && MessageParagraph}
      <div className="m-4 flex flex-col items-center">
        </div>
      {!isOwner && MessageParagraph}
    </div>
  );
}

```

useSocket.tsx

```

import { socket } from "@pages/api/socket";
import { useEffect, useState } from "react";

const useSocket = (userId: number) => {
  const [socketInstance, setSocketInstance] = useState(
    socket({
      userId,

```

```
    })  
  );  
  const [isConnected, setIsConnected] = useState(socketInstance.connected);  
  
  useEffect(() => {  
    function onConnect() {  
      console.log('Conectado ao servidor Socket.IO');  
      setIsConnected(true);  
    }  
  
    function onDisconnect() {  
      console.log('Desconectado do servidor Socket.IO');  
      setIsConnected(false);  
    }  
  
    socketInstance.on("connect", onConnect);  
    socketInstance.on("disconnect", onDisconnect);  
  
    return () => {  
      socketInstance.off("connect", onConnect);  
      socketInstance.off("disconnect", onDisconnect);  
    };  
  }, [socketInstance, userId]);  
  
  return { socketInstance, isConnected };  
};
```

```
export default useSocket;
```

Socket.ts

```
import { io, ManagerOptions } from "socket.io-client";  
  
export const socket = (query?: ManagerOptions["query"]) =>  
  io("http://localhost:3333", { query });
```

Não deixe de praticar!

SAIBA MAIS

Quando aplicamos o conceito de BFF em Next.js, um framework React popular para desenvolvimento web, estamos essencialmente falando sobre a configuração de um servidor intermediário que trata especificamente da lógica e das necessidades de dados do frontend construído com Next.js.

Este servidor BFF atua como uma camada entre o frontend e outros serviços de backend ou APIs, permitindo uma integração mais limpa e direcionada.

Vantagens de usar BFF com Next.js

- Personalização para o front-end: o BFF pode ser otimizado para as necessidades específicas do frontend, oferecendo uma API personalizada que reduz a complexidade e melhora o desempenho ao consumir dados.
- Segurança aprimorada: ao intermediar as requisições entre o cliente e os serviços de backend, o BFF pode implementar lógicas de autenticação e autorização, protegendo os serviços de backend contra acessos diretos indesejados.
- Desacoplamento: facilita a evolução e manutenção do frontend e do backend de forma independente, permitindo atualizações em um lado sem necessariamente impactar o outro.
- Otimização de desempenho: o BFF pode agregar, filtrar e ajustar dados de várias fontes de backend antes de enviá-los ao frontend, reduzindo o número de chamadas de rede e melhorando a experiência do usuário.

Implementação de um BFF em Next.js

A implementação de um BFF em um projeto Next.js pode ser realizada de várias maneiras, dependendo dos requisitos específicos do projeto. Algumas abordagens incluem:

- API Routes do Next.js: utilizar as rotas API do Next.js para criar endpoints que atuem como um BFF. Esses endpoints podem processar dados, realizar chamadas a serviços externos, e retornar respostas formatadas para o frontend.
- Servidores Express/Node.js personalizados: integrar um servidor Express ou outro servidor Node.js no projeto Next.js para lidar com lógicas mais complexas de BFF, oferecendo uma separação clara entre o servidor de aplicação frontend e a lógica de BFF.
- Middleware de edge: com a introdução de funcionalidades de edge computing em plataformas como Vercel (que hospeda muitos aplicativos Next.js), é possível implementar lógicas de BFF em edge functions, que operam mais próximas ao usuário, reduzindo latência.

Aprofundando mais os tópicos sobre o uso de BFF com NextJs

Sobre personalização de API, podemos dizer que usar um BFF (Backend for Frontend) em projetos Next.js é uma estratégia poderosa que foca na criação de interfaces de programação de aplicativos (APIs) que são otimizadas especificamente para as necessidades da interface do usuário (UI) do Next.js. Essa abordagem permite uma comunicação mais eficiente e direcionada entre o frontend e os serviços de backend, resultando em uma experiência de usuário aprimorada, desempenho melhorado e desenvolvimento frontend simplificado.

Vamos explorar como essa personalização de API pode ser alcançada e os benefícios que ela oferece.

Otimização de chamadas de API

Um dos principais benefícios de usar um BFF para personalizar APIs em Next.js é a capacidade de otimizar chamadas de API. O BFF pode combinar dados de várias fontes de backend em uma única resposta de API que contém exatamente o que o frontend precisa, eliminando chamadas de rede desnecessárias e reduzindo a sobrecarga de dados. Isso não apenas melhora o desempenho, mas também simplifica a lógica do lado do cliente.

Redução de complexidade do frontend

Com um BFF, a complexidade de interagir com múltiplos microsserviços ou sistemas de backend é abstraída para o servidor BFF. O frontend apenas precisa fazer requisições para o BFF, que cuidará da lógica de agregação, transformação e filtragem dos dados. Isso simplifica significativamente o desenvolvimento frontend, permitindo que os(as) desenvolvedores(as) se concentrem na experiência do usuário.

Desacoplamento e flexibilidade

O desacoplamento e a flexibilidade são aspectos fundamentais no uso de um Backend for Frontend (BFF) em projetos Next.js, proporcionando uma arquitetura mais robusta, escalável e adaptável às mudanças. Este padrão permite que equipes de desenvolvimento tratem de maneira eficaz a complexidade inerente aos sistemas modernos, especialmente aqueles que servem múltiplos clientes frontend (como web, mobile, e outros dispositivos) a partir de uma variedade de serviços de backend.

Vamos explorar como o BFF contribui para o desacoplamento e a flexibilidade em projetos Next.js?

Desacoplamento entre frontend e backend

O uso de um BFF cria uma camada intermediária específica entre o frontend e os serviços de backend. Isso significa que o frontend não precisa se comunicar diretamente com vários serviços de backend, cada um possivelmente utilizando diferentes protocolos, padrões de dados e mecanismos de autenticação. Em vez disso, o frontend interage apenas com o BFF, que é projetado para atender às suas necessidades específicas. Esse desacoplamento facilita a manutenção e a atualização tanto do frontend quanto dos serviços de backend, pois as alterações em um lado não exigem modificações diretas no outro, desde que o contrato de API estabelecido pelo BFF seja mantido.

Flexibilidade na integração de serviços

Um BFF oferece uma grande flexibilidade na integração de diferentes serviços de backend. Ele pode se comunicar com diversos sistemas - seja um banco de dados, um serviço de terceiros, um microserviço interno ou até mesmo outro BFF - e agir como um agregador ou orquestrador de dados. Isso permite que pessoas desenvolvedoras construam funcionalidades complexas de maneiras mais simples e centralizadas, adaptando-se facilmente a novos requisitos de negócios ou mudanças nos sistemas de backend existentes.

Personalização para diferentes clientes

A arquitetura BFF brilha especialmente quando há necessidade de servir múltiplos clientes frontend com requisitos distintos. Ao invés de forçar todos os clientes a se adaptarem a uma única API genérica de backend, é possível criar BFFs personalizados que fornecem exatamente o que cada cliente precisa. Isso não apenas otimiza a carga de dados e melhora a performance, mas também permite experiências de usuário customizadas para cada tipo de cliente, seja um navegador web, um aplicativo móvel ou outro dispositivo.

Agilidade e velocidade de desenvolvimento

O desacoplamento proporcionado pelo BFF aumenta a agilidade e a velocidade de desenvolvimento. As equipes de frontend podem trabalhar de maneira mais independente, focando na experiência do usuário e na implementação de novas funcionalidades sem se preocupar com os detalhes complexos dos sistemas de backend. Da mesma forma, as equipes de backend podem focar na lógica de negócios e na otimização dos serviços, sem serem impactadas diretamente pelas necessidades específicas de cada cliente frontend.

Simplificação do gerenciamento de estado e cache

Ao centralizar a lógica de comunicação com os serviços de backend, o BFF também simplifica o gerenciamento de estado e o cache dos dados. Isso permite implementar estratégias eficientes de cache no lado do servidor, reduzindo a carga sobre os serviços de backend e melhorando a resposta aos usuários finais.

Implementação via API Routes e middleware de edge

A implementação de um backend for frontend (BFF) em projetos Next.js pode ser alcançada de duas maneiras principais: por meio de API Routes ou utilizando middleware de edge. Ambas as abordagens têm suas próprias vantagens, desvantagens e diferenças significativas, dependendo dos requisitos do projeto, da escala, da performance desejada e da complexidade da infraestrutura.

Implementação via API Routes

Vantagens

- Integração simplificada: API Routes são integradas diretamente no Next.js, facilitando a implementação de um BFF sem a necessidade de uma infraestrutura separada.
- Desenvolvimento unificado: permite que pessoas desenvolvedoras trabalhem dentro do mesmo projeto Next.js para frontend e backend, mantendo uma base de código coesa e simplificando o processo de desenvolvimento.
- Facilidade de uso: aproveita o conhecimento existente do Next.js e do ecossistema Node.js, tornando a curva de aprendizado para implementação de API Routes relativamente suave.

Desvantagens

- Escalabilidade: embora seja conveniente para projetos menores ou de médio porte, pode se tornar um gargalo para aplicações de alta demanda devido ao processamento das requisições ser feito no mesmo ambiente que serve o frontend.
- Menor separação de preocupações: a mistura de lógica de frontend e backend no mesmo projeto pode levar a uma base de código mais complexa e potencialmente mais difícil de manter e escalar, especialmente em equipes grandes.

Implementação via middleware de edge

Vantagens

- Desempenho e latência: o middleware de edge é executado mais próximo do usuário final, em servidores distribuídos globalmente, o que pode significativamente reduzir a latência e melhorar o desempenho das aplicações.
- Escalabilidade: ideal para aplicações de alta demanda, pois o processamento é distribuído por meio de uma rede de edge, aliviando a carga no servidor de origem.
- Segurança aprimorada: o processamento de requisições em edge pode adicionar uma camada extra de segurança, filtrando tráfego indesejado ou malicioso antes que ele alcance a infraestrutura de backend principal.

Desvantagens

- Complexidade de implementação: requer um entendimento mais profundo de redes de entrega de conteúdo (CDN) e conceitos de edge computing, podendo apresentar uma curva de aprendizado mais acentuada.
- Custo: embora o custo possa ser competitivo, para projetos de grande escala que utilizam intensivamente recursos de edge pode haver um aumento nos custos operacionais em comparação com a hospedagem tradicional ou o uso exclusivo de API Routes.
- Limitações de recursos: dependendo do provedor de edge computing, pode haver limitações em termos de tempo de execução, memória disponível e capacidades de computação, o que pode impactar a complexidade das operações que podem ser realizadas.

Principais diferenças

- Localização da execução: enquanto as API Routes são executadas no servidor de origem (ou em um ambiente de servidor em cloud), o middleware de edge é executado em pontos de presença distribuídos globalmente.

- Performance e latência: middleware de edge tende a oferecer melhor desempenho e menor latência devido à sua proximidade com o usuário final, ao contrário das API Routes, onde o desempenho pode ser impactado pela distância entre o servidor de origem e o usuário.
- Complexidade e custo: a implementação via middleware de edge pode ser mais complexa e potencialmente mais cara, especialmente para projetos de grande escala, enquanto as API Routes são mais simples e mais integradas ao ecossistema Next.js.

Ambas as abordagens têm seu lugar dependendo das necessidades específicas do projeto. Para aplicações menores ou projetos com requisitos de desenvolvimento mais simples, as API Routes podem oferecer uma solução rápida e eficaz. Por outro lado, para aplicações que demandam alta performance, baixa latência global e escalabilidade, o middleware de edge pode ser a escolha preferencial, apesar de sua complexidade e potencial custo adicional.

Gerenciamento de estado e cache

O gerenciamento de estado e cache no contexto de Backends for Frontends (BFFs) em projetos Next.js é um aspecto crucial para otimizar o desempenho e a experiência do usuário. O BFF atua como uma camada intermediária entre o frontend e os serviços de backend, não apenas personalizando a comunicação de dados para as necessidades específicas do frontend, mas também oferecendo oportunidades únicas para implementar estratégias de cache e gerenciamento de estado eficientes. Vamos explorar como essas funcionalidades podem ser aproveitadas e os benefícios que trazem.

Gerenciamento de estado

O gerenciamento de estado em um BFF é sobre manter um registro dos dados necessários para atender às solicitações do frontend de forma eficiente. Isso pode incluir informações sobre sessões de usuário, preferências, dados de autenticação e mais. A principal vantagem de gerenciar o estado no BFF é a capacidade de otimizar

as interações com os serviços de backend, reduzindo a necessidade de solicitações redundantes e melhorando a velocidade de resposta para o usuário final.

- **Centralização:** o BFF permite centralizar o gerenciamento de estado para vários serviços de backend, proporcionando um ponto de controle único para gerenciar autenticações, sessões, e outros estados relevantes.
- **Personalização:** com o estado gerenciado no BFF, é possível personalizar a lógica de negócios e as respostas de API com base no estado do usuário ou em outras condições específicas, melhorando a experiência do usuário.

Cache

O cache no BFF é uma estratégia poderosa para otimizar o desempenho e reduzir a carga nos serviços de backend. Ao armazenar temporariamente cópias de dados frequentemente acessados ou computacionalmente intensivos, o BFF pode servir respostas rápidas para solicitações subsequentes, melhorando significativamente a experiência do usuário.

- **Redução de latência:** o cache de dados no BFF pode reduzir drasticamente a latência, servindo dados de cache em vez de realizar uma nova chamada de backend a cada solicitação.
- **Diminuição da carga no backend:** ao servir dados do cache, o BFF diminui a quantidade de solicitações aos serviços de backend, o que é especialmente valioso durante picos de tráfego.

Estratégias de cache

- **Cache de API:** armazenar respostas de API no BFF para solicitações frequentes ou dados que mudam raramente. Isso é útil para dados como listagens de produtos, artigos, ou informações que são atualizadas em intervalos regulares.
- **Invalidação de cache:** implementar uma estratégia eficaz de invalidação de cache é crucial para garantir que os usuários recebam dados atualizados sem comprometer o desempenho. Isso pode incluir técnicas

como cache com tempo de vida (TTL), invalidação baseada em eventos, ou estratégias de cache condicional.

- Cache personalizado: utilizar o BFF para criar regras de cache personalizadas com base no tipo de usuário, localização geográfica, ou outros fatores específicos, otimizando ainda mais a experiência do usuário.

EXEMPLO

O QUE VOCÊ VIU NESTA AULA?

Você aprendeu sobre o uso de backend for frontend em projetos Next.js, explorando as vantagens que nos são proporcionadas e aumentando a eficiência da nossa aplicação. Também aprendeu sobre boas práticas, vantagens e desvantagens do uso de BFF e como é possível utilizá-lo em nossa aplicação.

Revisitamos alguns componentes criados anteriormente e vimos como utilizávamos as camadas de comunicação entre front e backend, e em seguida fizemos uma camada de comunicação para efetuarmos a comunicação em realtime, criamos hooks e configuramos o socket-io na nossa aplicação.

REFERÊNCIAS

ADEL. BFF – Backend for Frontend Design Pattern with Next.js. 2021. Disponível em: <https://dev.to/adelhamad/bff-backend-for-frontend-design-pattern-with-nextjs-3od0>.

Acesso em: 28 jun. 2024.

NEXT.JS. Documentação Oficial. [s.d.]. Disponível em: <https://nextjs.org/docs/>.

Acesso em: 28 jun. 2024.

VERCEL. Vercel. [s.d.]. Disponível em: <https://vercel.com/blog/>. Acesso em: 28 jun. 2024.

PALAVRAS-CHAVE

NextJs. BFF. Socket-io.

EMSE

POSTECH